

AI 使用说明情况

HW1 挑战内容 6

1. 使用的 AI 模型

CodeBuddy 智能编码助手（内置大模型，具备沙箱终端、文件读写与联网检索能力），全程在同一工作环境中以"边执行边核对"的方式协作。

2. 在哪些环节使用 AI、使用了什么提示词（真实摘录）

环节	提示词 / 指令（摘录）	AI 的实际作用
环境部署	"启动 SGLang CPU 后端报 ModuleNotFoundError: vllm，但代码里有 is_cpu() 分支，帮我查 CPU 后端的正确启动方式"	顺着 sglang/srt/utils/common.py 的 is_cpu() 找到 SGLANG_USE_CPU_ENGINE=1 开关，给出启动命令
故障定位	"scheduler 初始化阶段 exit code -9，free 显示 123G 内存却像 OOM，帮我分析"	提示检查 cgroup 而非宿主视角，用 memory.events 的 oom_kill 计数确认根因
实验设计	"帮我写一组对照实验：32 条请求、并发 8、共享前缀 vs 分散前缀，用 /generate 流式接口，指标要 TTFT/TPOT/命中率"	产出 run_benchmark.py 框架（预热→flush→预热请求→并发回放）
源码阅读	"沿 TokenizerManager → Scheduler → RadixCache → worker 的主线帮我定位 v0.5.14 的关键函数与行号"	给出各环节函数与文件:行号清单，并解释调用关系
文档与可视化	"把实验结果整理成对照表，并画一张请求主流程图"	生成表格与 SVG 流程图初稿

3. AI 如何帮助我学习

- **降低上手成本**：推理框架的部署细节多（依赖、编译后端、环境变量），AI 把"报错 → 源码位置 → 修复"的闭环缩短到分钟级。
- **充当源码导读**：v0.5.14 的调度与缓存代码量很大，AI 能快速给出函数级地图，我再精读关键分支，比盲读快得多。
- **协助把直觉变成可检验的实验**："前缀复用只影响 TTFT 不影响 TPOT"这一预期，是在 AI 协助下变成了两组对照实验并验证的。

4. 是否被 AI 误导（真实案例）

- **误判接口语义**：AI 依据单次观察认为 /flush_cache 成功时返回不含括号提示句，脚本据此判断"永远失败"；对照源码（http_server.py:841）后发现成功返回 200 且固定附带说明句。**教训：接口语义以源码为准，不以单次输出为准。**
- **随机性假设错误**：AI 生成的"分散前缀"构造把语料循环拼接，实际各组前缀相同，分散组命中率高达 74.9%；后来加入"最长公共前缀自检"才暴露该问题。**教训：对负载构造要做分布层面的断言，不能只看能不能跑通。**
- **危险命令**：AI 建议的 pkill -f sglang.launch_server 匹配到了自身 shell 导致会话被杀；改为把匹配串放进独立脚本执行。**教训：模糊匹配的批量 kill 命令要先想清楚匹配域。**

总体而言：AI 显著加速了工程与阅读环节，但它给出的"经验假设"必须经过实测/源码双重校验；本作业中所有提交的数据都来自脚本的真实运行结果。